

SDAI HIMSHIKHAR 2026

APIPhoenix

AI-Powered API Documentation & Testing Portal

Your APIs — documented, sampled, and battle-tested in seconds.

TEAM PHOENIX

Sri Varshith Vaitla · Nekkanti Bhavita · Pula Nikarika
Sai Ankith Reddy Kancharla



The problem — and who it hurts

APIs run everything. Their documentation rots first.



Docs drift from code

Hand-written docs go stale the moment an endpoint changes — developers reverse-engineer behavior instead of reading it.



Boilerplate everywhere

Every integration re-creates the same cURL / JS / Python sample requests and the same basic test cases by hand.



Errors without answers

A raw 422 or 409 tells a junior developer nothing about what to actually do next.

Real-world impact: student teams, junior developers and small startups get readable docs, runnable samples, a test report, and a concrete next step — from one upload.



Starter data — authored, not borrowed

data/endpoints.json · schema: method, path, request_body, response_body, auth_required, error_codes

49

realistic endpoint records

5

fictional services, fully
reproducible generator

Service	Domain	Endpoints
ShopStack	E-commerce — auth, catalog, cart, orders, payments	16
RideLink	Mobility — estimates, rides, drivers, surge, wallet	10
MediTrack	Healthcare — patients, appointments, labs, vitals	9
EduSphere	Ed-tech — courses, quizzes, certificates, doubts	7
PayBridge	Fintech — intents, refunds, webhooks, UPI	7

Deliberate variety: public vs. authenticated · path & query params · domain business errors (402, 409, 423, 429)



System workflow

Clean path from input to actionable output — it never stops at code execution.



Stack: single-file HTML + Three.js 3D front-end · optional FastAPI + SQLite backend · Claude API (claude-sonnet-4-6)



Where the AI lives — and why

Language judgment from the model, structure and verification from the system.

AI documentation generator

JSON contract → structured
Markdown a human can read:
overview, params, responses, usage
notes.

AI test-case generator

5–7 cases per endpoint as strict
JSON — positive, auth, validation,
negative, edge, contract.

AI Error Doctor

Any pasted error → diagnosis, likely
causes, and a numbered “what to do
next”.



The Phoenix fallback engine (our reliability innovation)

Every AI call falls through to a deterministic rule engine on failure — same three tasks, driven by the endpoint schema and an HTTP knowledge base. Output is always produced; every result is badged “⚡ Claude AI” or “🔧 Phoenix local”, so provenance is transparent and the demo can never break offline.



Prototype tour

A 3D endpoint constellation greets the user; four tools do the work. (Live screenshots in /screenshots.)

ENDPOINTS

GET /products

POST /orders

PUT /products/{id}

PATCH /stock

DELETE /items/{id}

POST /rides

GET /reports/{id}

Documentation

AI-written docs + contract snapshot tables,
exportable to Markdown

Playground

cURL / JavaScript / Python samples · simulate
success & failure responses

Test Lab

One-click suite generation · animated run ·
PASS/FAIL per case

Error Doctor

Paste any error → plain-English diagnosis + next
steps



Results & validation

The Test Lab is contract-grounded: a case passes only if the endpoint's own contract declares that behavior.

49/49

endpoints fully documented by
either engine

6

test-case archetypes
generated per suite

3

sample-request languages
(cURL · JS · Python)

1-click

hand-off from failed test to
Error Doctor

Sample run – POST /api/v1/orders (ShopStack)

PASS Happy path returns success

expect 2xx

PASS Rejects missing bearer token

expect 401

PASS Rejects malformed body

expect 422

PASS Payment failure handled

expect 402

FAIL Wrong method returns 405

expect 405

4/5

passed — the 405 case is
honestly reported, not rubber-
stamped



Limitations & responsible use

Stated honestly — this is a prototype with clear boundaries.

Limitations

- Test execution is contract-level simulation, not live HTTP against a deployed server
- AI output can be imprecise or over-confident — generated docs are drafts
- Single-file portal keeps state in memory; persistence needs the FastAPI + SQLite path

Responsible use

- Never paste secrets, tokens, or personal/health data — all bundled data is fictional
- Every output is badged by engine (AI vs. local) for transparent provenance
- AI calls are minimal and user-initiated — one per generation click, no background use



Future improvements

The roadmap after Himshikhar.

1

Live test execution

Run suites against real base URLs with CORS-aware proxying and auth handling

2

OpenAPI 3.1 in/out

Import Swagger specs directly; export generated docs back as OpenAPI

3

Doc-drift CI hook

Fail the build when the code's contract diverges from published docs

4

Schema assertions

Runner validates response bodies against schemas, not just status codes

APIPhoenix

From raw route definitions to docs, samples, tests and next steps —
one clear workflow, AI where language matters, a rule engine where reliability matters.

49

endpoints

5

services

4

AI-assisted tools

100%

offline availability

Thank you · Team Phoenix — Sri Varshith Vaitla · Nekkanti Bhavita · Pula Nikarika · Sai Ankith
Reddy Kancherla · SDAI Himshikhar 2026